Labo 4 - Programmation d'une communication sur le bus CAN

1. Objectifs pédagogiques

- Apprendre les principes de programmation d'un contrôleur CAN.
- Observer pratiquement les caractéristiques de fonctionnement de CAN.
- Mesurer les performances d'une liaison CAN.
- Être capable de mettre en œuvre efficacement CAN dans des solutions électroniques.

2. Travaux préparatoires

Parcourir la documentation technique fournie en annexe 1 pour comprendre globalement le fonctionnement et les possibilités offertes par le contrôleur CAN MCP2515.

Prendre connaissance des fonctions fournies qui simplifieront les tâches de programmation en évitant de gérer dans le détail les registres du contrôleur (annexe 2).

3. Présentation du matériel et du logiciel fournis

3.1. *Matériel*

Pour mettre en œuvre le bus de terrain CAN, il s'agit d'utiliser deux cartes à microcontrôleur Arduino MEGA 2560, équipées de Shield CAN construits autour des circuits MCP2515 et MCP2551.

Ce système à microcontrôleur dispose des éléments suivants :

- 8 entrées tout ou rien 24 V (seuil 3.3 V).
- 8 sorties tout ou rien de type PNP (niveau 12 V).
- 4 entrées analogiques 0..5 V.
- 4 sorties analogiques 0..5 V.

Pour programmer la carte, l'outil de développement Arduino version 1.7.8 (ou supérieure) sera utilisé, ainsi qu'un câble USB pour relier le PC à l'arduino.

Le bus CAN disponible sur le connecteur Sub-D 9 pôles doit être connecté entre cartes pour permettre la communication. Le brochage respecte la norme CanOpen ci-dessous.

Note : Pour permettre une liaison avec de multiples stations, les résistances de terminaison de 120 ohms intégrées sur les Shield CAN ont été déconnectées. Il faut donc mettre en place des résistances de terminaison sur le bus là où c'est nécessaire.

Durant le câblage, il peut arriver que deux ports du Sub-D soient incompatibles (mâle vers mâle ou femelle vers femelle). Des adapteurs sont mis à disposition pour permettre la liaison entre deux connecteurs du même genre.

3.2. *Logiciel*

La programmation à bas niveau des registres du microcontrôleur est assez fastidieuse et sort du cadre de ce travail de laboratoire. Pour permettre de se focaliser sur la partie spécifique à la mise en œuvre de CAN, des fonctions de base réalisant la gestion à bas niveau des entrées/sorties analogique et de la carte CAN sont fournies dans les fichiers « BusterUtils.h » et « BusterUtils.c ». Pour les entrées sorties tout-ou-rien, il faut faire appel tout simplement aux fonctions de base standards fournies par la bibliothèque Arduino :

`digitalWrite(pin, value)`

Parameters

pin: the pin number

value: HIGH or LOW

Returns

none

`digitalRead(pin)`

Parameters

pin: the number of the digital pin you want to read (int)

Returns

HIGH or LOW

Les numéros des IO à utiliser sont définis par des symboles à l'aide de la directives #define dans « BusterUtils.h », comme par exemple :

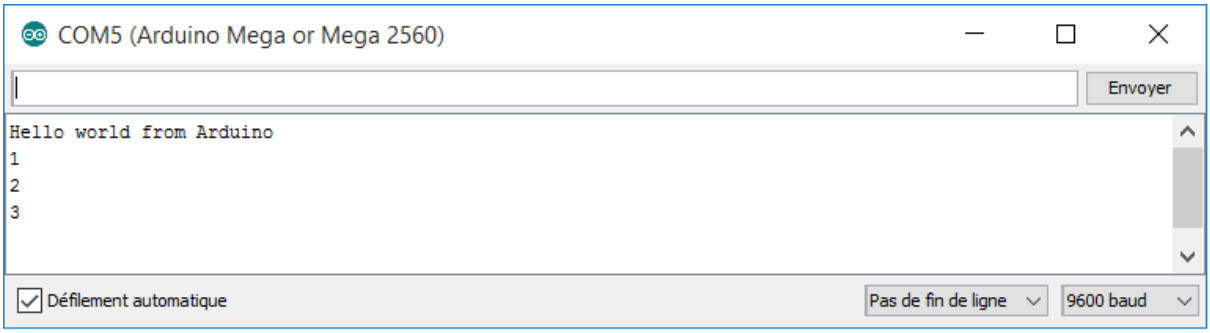
```
// DIGITAL INPUT
#define BUSTER_PIN_IN_0  40
. . .
#define BUSTER_PIN_IN_7  35
```

Consulter le listing du fichier BusTer.h imprimé en annexe, ou directement dans le fichier source fourni. La documentation et les fichiers sources de base à utiliser se trouvent dans le dossier réseau du laboratoire BusTer.

Pour le débogage, utiliser les fonctions Serial.begin, Serial.print et Serial.println pour afficher des valeurs ou des messages dans la console. Exemple :

Consulter si nécessaire la documentation Arduino à ce sujet.

Truc : Pour ouvrir la console durant l'exécution, il suffit d'appuyer simultanément sur « ctrl + maj + m »



4. Tâches à effectuer

4.1. *Programmation*

Dans un premier temps, il s'agit d'écrire une application permettant de tester le protocole CAN, puis de procéder à des mesures et des observations de fonctionnement. Le cahier des charges de cette application est décrit ci-dessous. Il doit être respecté précisément.

Les fichiers fournis dans le dossier du laboratoire BusTer « BusTerUtils.h » et « BusTerUtils.cpp » doivent être placés dans le dossier de votre application Arduino.

Pour l'application développée dans le cadre de ce laboratoire, il faut inclure les fichiers suivants au début du programme :

```
#include <Wire.h>
#include <SPI.h>
#include "BusTerUtils.h"
```

En raison des règles de développement propres à l'environnement Arduino, ces fichiers doivent être inclus même si vous ne faites pas appel explicitement aux fonctions contenues, car elles sont requises pas les fonctions de BusTerUtils.cpp.

Remarque importante : Pour ajuster la configuration des entrées/sorties, appeler la fonction BusTer_InitInputOutput() en tout premier dans la fonction setup() de votre application Arduino.

4.1.1. Objectif global

Le but est d'implémenter un protocole de communication sur CAN qui transmet les informations entre 2 cartes CAN, de telle sorte que les valeurs des entrées tout ou rien lues sur une carte soient écrites sur les sorties tout ou rien de l'autre carte et inversement.

Le même principe sera appliqué aux entrées et sorties analogiques.

Ainsi, un signal injecté sur une entrée d'une carte sera répété sur la sortie correspondante de l'autre carte, ce qui permet en quelque sorte de faire transiter les signaux par le bus CAN.

Comme le protocole est totalement symétrique, le même programme sera chargé dans les 2 cartes à microcontrôleur.

4.1.2. Format des messages

Le protocole à développer utilise le format de messages CAN2.0A, avec un champ d'arbitration constitué de 11 bits.

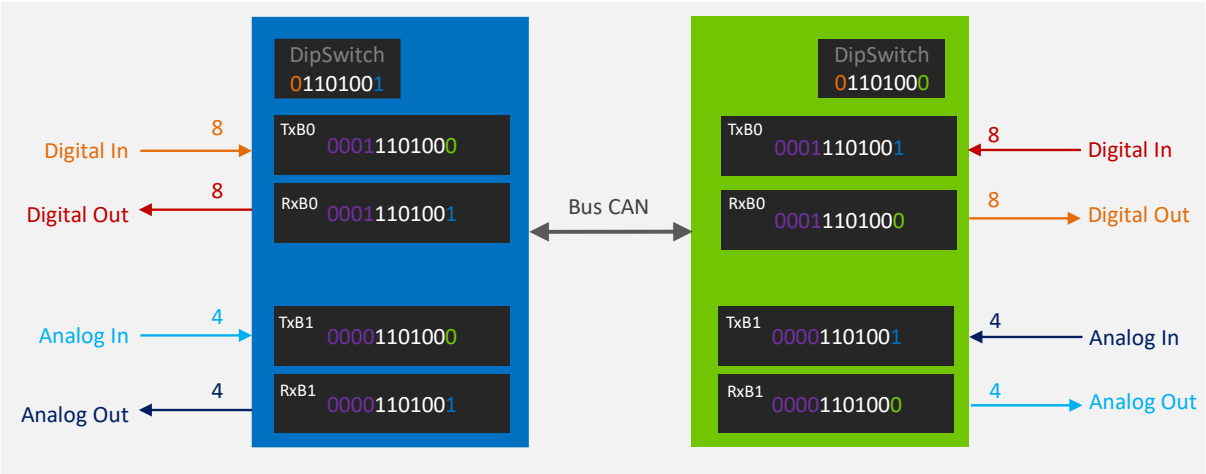
Les 7 bits de 0 à 6 constituent des bits d'adresse.

Les 4 bits de 7 à 10 constituent le numéro de message.

Les numéros de message suivants sont définis :

4 bits	Valeur hexa dans message	Données
0000	0x000 (+ 7 bits d'adresse) En binaire : 000 0xxx xxxx	Message contenant 4 valeurs analogiques (8 octets) : AIN0_LOW, AIN0_HIGH, AIN1_LOW, AIN1_HIGH, AIN2_LOW, AIN2_HIGH, AIN3_LOW, AIN3_HIGH
0001	0x080 (+ 7 bits d'adresse) En binaire : 000 1xxx xxxx	Message contenant 1 port 8 bits d'entrées/sorties tout ou rien (1 octet).

Un nouveau message ne doit être envoyé par la carte microcontrôleur que si les valeurs des entrées correspondantes ont changé depuis le précédent message envoyé. On évite ainsi d'envoyer des nouveaux messages s'il l'état des entrées n'a pas évolué.



4.1.3. Exigence de développement

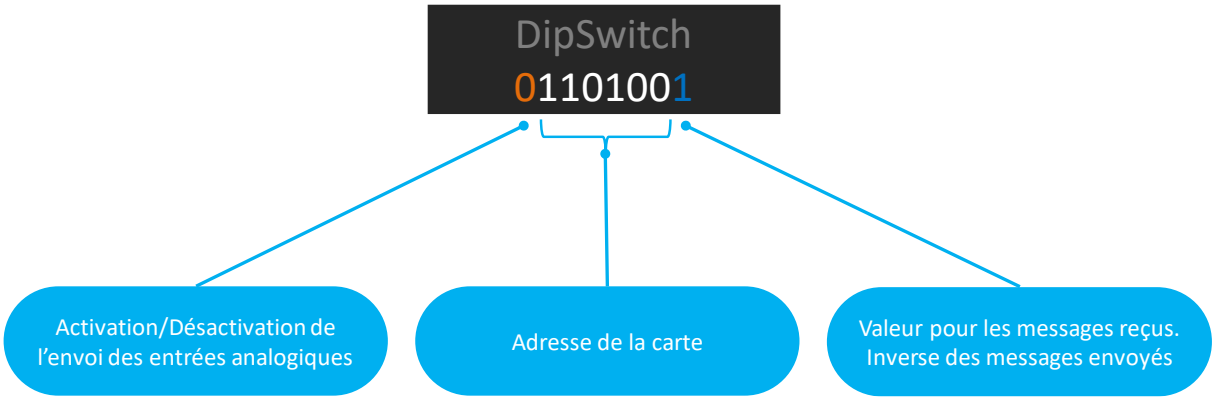
On demande explicitement d'utiliser le registre de réception Rxb0 pour les valeurs tout-ou-rien, et Rxb1 pour les valeurs analogiques.

Dans ce but, utiliser les fonctions de masque et de filtre du MCP2515, afin de garantir que les messages avec l'identifiant propre aux messages de type tout-ou-rien soient placés automatiquement dans le registre Rxb0, et les messages pour l'analogique dans Rxb1.

Note : tous les registres de filtres doivent être définis, sinon, ils provoquent la capture de message d'après leur valeur d'initialisation.

4.1.4. Configuration par DIP SWITCH

Un ensemble de 8 interrupteurs « DIP SWITCH » se trouve sur la carte.



Les interrupteurs 1 à 8 seront utilisés par le programme.

DIP 1	Ce bit correspond au bit d'adresse de poids faible pour les messages à recevoir. L'inverse de ce bit correspond au premier bit d'adresse pour les messages à envoyer. Ceci implique qu'il n'y a que ce bit qui change pour l'adresse entre les deux cartes jumelées.
DIP 2 - 7	6 bits d'adresse. Le DIP7 correspond au bit d'adresse de poids le plus fort.
DIP 8	Si ce bit est à 1, la carte CAN doit envoyer les valeurs analogiques à chaque changement. Si ce bit est à 0, la carte ne doit pas gérer les valeurs analogiques, ni à l'émission, ni à la réception. Dans tous les cas, la carte doit envoyer les valeurs des entrées tout ou rien.

Exemple :

Carte 1 : Valeur du Dip Switch 0x3A. Cette carte reçoit des messages contenant 0x3A dans les bits 0 à 6 du champ d'arbitration, elle envoie des messages contenant 0x3B.

Carte 2 : Valeur du Dip Switch 0x3B. Cette carte reçoit des messages contenant 0x3B dans les bits 0 à 6 du champ d'arbitration, elle envoie des messages contenant 0x3A.

En conséquence, avec ces valeurs configurées sur les DIP Switch, les messages envoyés pas la carte 1 sont reçus par la carte 2 et inversement.

4.1.5. Autres fonctions

La carte comporte une led d'état bicolore rouge/verte. Pour faciliter l'observation du trafic, allumer la led verte depuis l'écriture du message à émettre dans le registre TXB jusqu'à la fin de son émission.

Allumer la led rouge si une erreur de bus survient. Après une pause de 100 ms, éteindre la led rouge et quittancer l'erreur automatiquement.

Pour exécuter une attente, utiliser la fonction standard delay(ms) de l'arduino.

4.2. *Mesures*

Une fois que le programme fonctionne conformément au cahier des charges, il s'agit de procéder à une série de mesures pour mettre en évidence le fonctionnement du bus Can.

Pour les premières mesures, configurer la vitesse du bus à 1MBPs.

4.2.1. Analyse de trame

Pour cette première étape, désactiver l'envoi de données analogiques.

Capturer une trame CAN (correspondant à des entrées sorties tout ou rien ayant toutes la même valeur) à l'oscilloscope.

- Intégrer l'oscillogramme au rapport.
- Mettre en évidence la structure du message CAN sur l'oscillogramme.
- Vérifier la fréquence de bit.

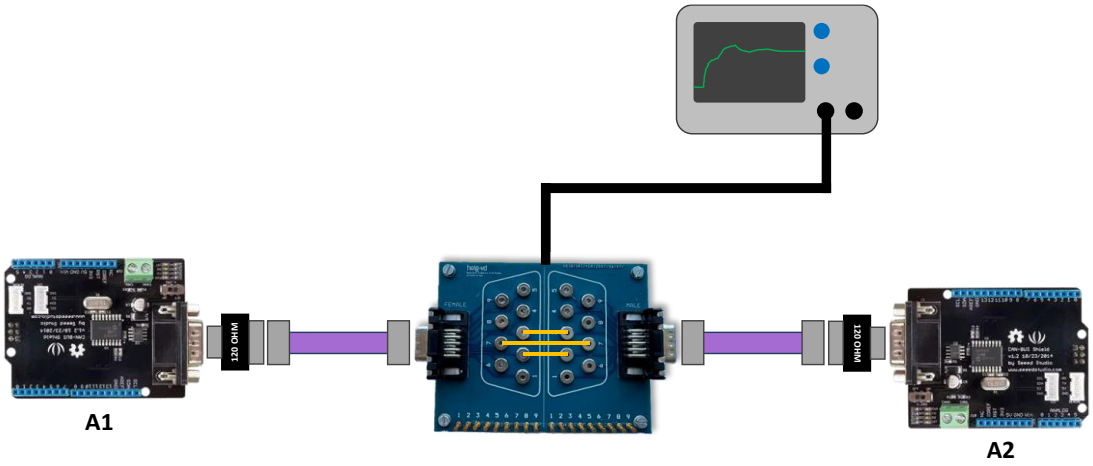


Figure 1 Exemple de câblage pour une mesure de la trame

4.2.2. Comportement en cas de problème de transmission

Débrancher la liaison CAN et provoquer l'envoi d'un message. Observer à l'oscilloscope l'activité sur le bus du côté émetteur.

Que constate-t-on ?

Cela correspond-il au comportement attendu ou annoncé ?

4.2.3. Latence et gigue

Sur un bus sans aucun trafic (messages analogiques désactivés), mesurer le temps de latence et la gigue entre le changement d'état d'une entrée et de la sortie correspondante sur la 2ème carte.

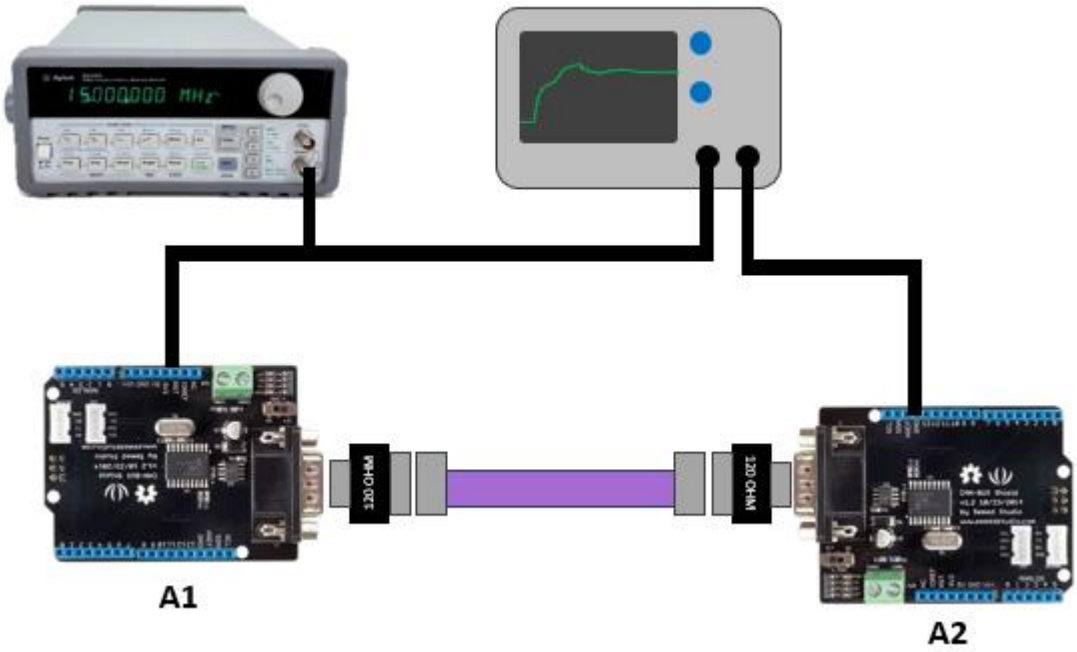


Figure 2 Exemple de câblage pour les mesures de gigues

4.2.4. Transmission de valeurs analogiques et trafic

Activer la transmission des valeurs analogiques dans un seul sens. Observer le trafic sur le bus en laissant l'entrée analogique à un potentiel fixe. Comment s'explique le trafic observé ?

4.2.5. Transmission d'un signal sinusoïdal

Injecter un signal sinusoïdal sur l'entrée analogique d'une carte. Visualiser ce signal à l'oscilloscope ainsi que le signal délivré en sortie sur l'autre carte.

D'après le signal de sortie, déduire quel est l'intervalle entre 2 messages CAN reçus.

Commenter la forme du signal de sortie observé.

Mettre en évidence l'influence de la fréquence du signal sinusoïdal sur la qualité de la transmission.

4.3. *Charge du bus*

Pour les prochaines mesures, connecter les cartes avec un autre groupe d'étudiant. Le but est d'avoir deux pairs de carte qui communique sur le même bus CAN.

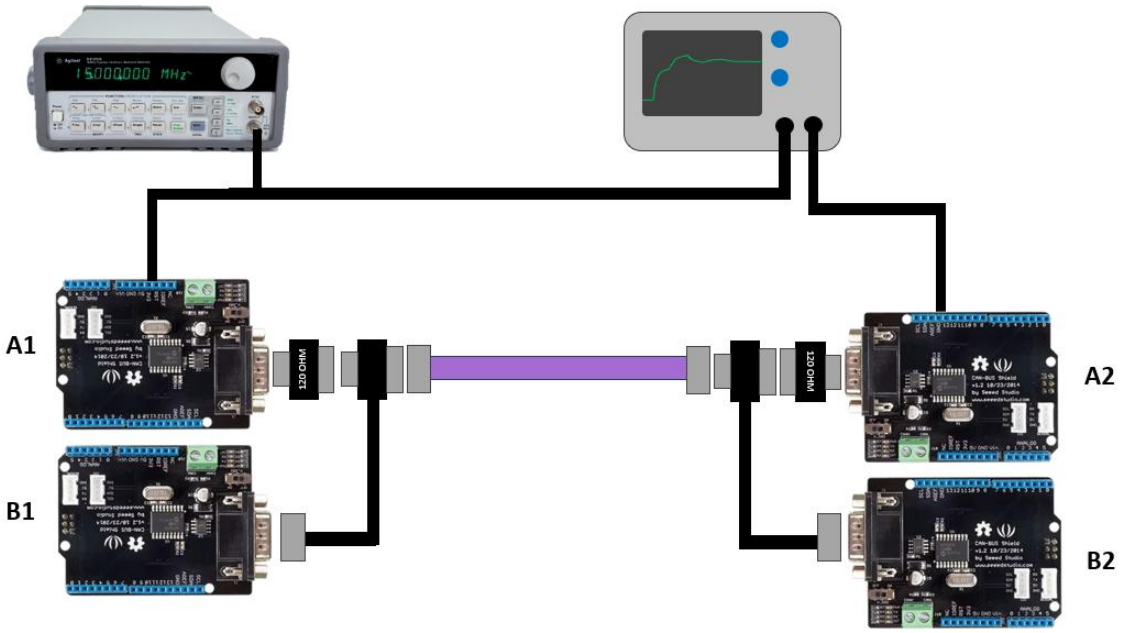


Figure 3 Exemple de câblage pour les mesures de charge du bus

Réduire la vitesse de communication de toutes les cartes à 200KBS.

Afin de simplifier le protocole de mesure, paramétrer les adresse comme ceci :

Carte	Adresse (dip1->dip7)
A1	22 (0110 100)
A2	23 (1110 100)
B1	72 (0001 001)
B2	73 (1001 001)

Vérifier la communication bidirectionnelle des deux paires de cartes. Une fois la communication validée, il est recommandé de commenter ou supprimer les Serial.Print() de votre code afin de garantir une bonne optimisation du code.

4.3.1. Latence et gigue

(Message analogique désactivé) Utiliser un signal carré à 500 Hz sur une entrée digital de la carte A1 et analyser la latence ainsi que la gigue. Ajouter ensuite le même signal sur une entrée de la carte B1 et analyser la latence et la gigue entre A1 et A2. Si l'augmentation de la gigue n'est pas significative, envoyer plus de message en utilisant des entrées de cartes pas encore utilisées ou en augmentant la fréquence.

4.3.2. Transmission d'un signal sinusoïdal sur un bus chargé

Activer la transmission analogique entre toutes les cartes, transmettre un signal sinusoïdal de 50 Hz de B2 vers B1 et l'observer. Transmettre ensuite un signal sinusoïdal de 50 Hz de B1 vers B2 et observer et commenter la différence qui apparait sur le premier signal. Ajouter un signal sinusoïdal à la même fréquence entre A2 et A1, puis observer ce qui se passe sur les sorties de B1 et B2. Finalement, ajouter un signal sinusoïdal de A1 vers A2 et analyser le comportement des sorties de B1et B2.

5. Travaux à rendre

5.1. *Liste des livrables*

- Les codes source du programme compilable.
- Le rapport imprimé, comportant une synthèse structurée des informations utiles obtenues au cours de ce laboratoire, ainsi que le listing du programme.
- Le fichier source du rapport, au format Word.

Les fichiers sont à placer dans le répertoire du laboratoire.

5.2. *Délai*

Le délai pour rendre la version imprimée et le fichier du rapport est fixé à 1 semaine après la fin de la dernière séance de laboratoire.

Annexe 1 – Le contrôleur CAN 2515 – Extrait du datasheet

MICROCHIP

MCP2515

Stand-Alone CAN Controller With SPI™ Interface

Features

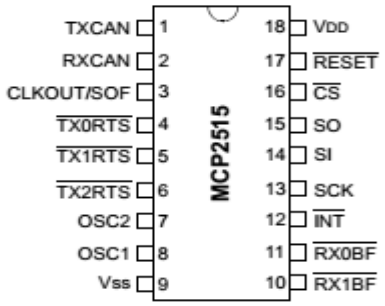
- Implements CAN V2.0B at 1 Mb/s:
  - 0 – 8 byte length in the data field
  - Standard and extended data and remote frames
- Receive buffers, masks and filters:
  - Two receive buffers with prioritized message storage
  - Six 29-bit filters
  - Two 29-bit masks
- Data byte filtering on the first two data bytes (applies to standard data frames)
- Three transmit buffers with prioritizaton and abort features
- High-speed SPI™ Interface (10 MHz):
  - SPI modes 0,0 and 1,1
- One-shot mode ensures message transmission is attempted only one time
- Clock out pin with programmable prescaler:
  - Can be used as a clock source for other device(s)
- Start-of-Frame (SOF) signal is available for monitoring the SOF signal:
  - Can be used for time-slot-based protocols and/or bus diagnostics to detect early bus degredation
- Interrupt output pin with selectable enables
- Buffer Full output pins configurable as:
  - Interrupt output for each receive buffer
  - General purpose output
- Request-to-Send (RTS) input pins individually configurable as:
  - Control pins to request transmission for each transmit buffer
  - General purpose inputs
- Low-power CMOS technology:
  - Operates from 2.7V – 5.5V
  - 5 mA active current (typical)
  - 1 µA standby current (typical) (Sleep mode)
- Temperature ranges supported:
  - Industrial (I): -40°C to +85°C
  - Extended (E): -40°C to +125°C

Description

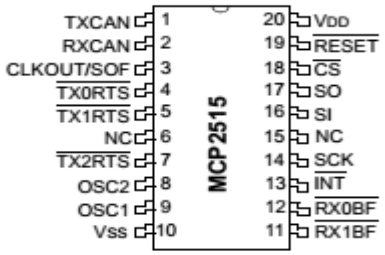
Microchip Technology's MCP2515 is a stand-alone Controller Area Network (CAN) controller that implements the CAN specification, version 2.0B. It is capable of transmitting and receiving both standard and extended data and remote frames. The MCP2515 has two acceptance masks and six acceptance filters that are used to filter out unwanted messages, thereby reducing the host MCUs overhead. The MCP2515 interfaces with microcontrollers (MCUs) via an industry standard Serial Peripheral Interface (SPI).

Package Types

18-Lead PDIP/SOIC



20-LEAD TSSOP



© 2005 Microchip Technology Inc. Preliminary DS21801D-page 1

1.0 DEVICE OVERVIEW

The MCP2515 is a stand-alone CAN controller developed to simplify applications that require interfacing with a CAN bus. A simple block diagram of the MCP2515 is shown in Figure 1-1. The device consists of three main blocks:

1. The CAN module, which includes the CAN protocol engine, masks, filters, transmit and receive buffers.
2. The control logic and registers that are used to configure the device and its operation.
3. The SPI protocol block.

An example system implementation using the device is shown in Figure 1-2.

1.1 CAN Module

The CAN module handles all functions for receiving and transmitting messages on the CAN bus. Messages are transmitted by first loading the appropriate message buffer and control registers. Transmission is initiated by using control register bits via the SPI interface or by using the transmit enable pins. Status and errors can be checked by reading the appropriate registers. Any message detected on the CAN bus is checked for errors and then matched against the user-defined filters to see if it should be moved into one of the two receive buffers.

1.2 Control Logic

The control logic block controls the setup and operation of the MCP2515 by interfacing to the other blocks in order to pass information and control.

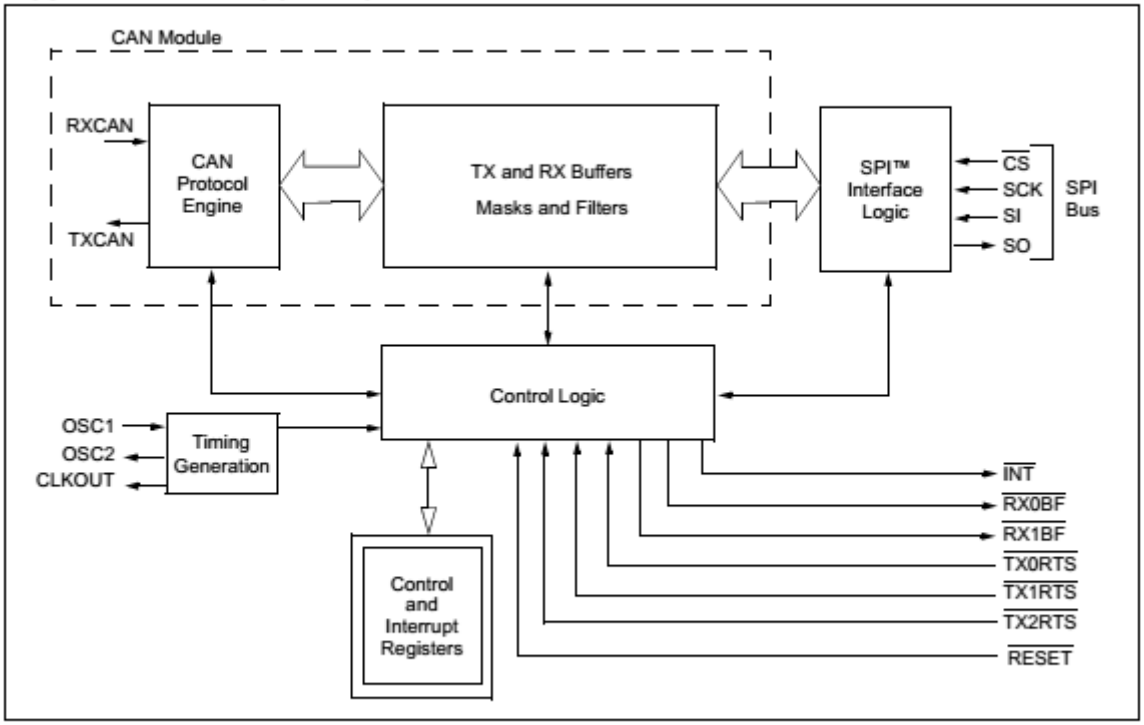
Interrupt pins are provided to allow greater system flexibility. There is one multi-purpose interrupt pin (as well as specific interrupt pins) for each of the receive registers that can be used to indicate a valid message has been received and loaded into one of the receive buffers. Use of the specific interrupt pins is optional. The general purpose interrupt pin, as well as status registers (accessed via the SPI interface), can also be used to determine when a valid message has been received.

Additionally, there are three pins available to initiate immediate transmission of a message that has been loaded into one of the three transmit registers. Use of these pins is optional, as initiating message transmissions can also be accomplished by utilizing control registers, accessed via the SPI interface.

1.3 SPI Protocol Block

The MCU interfaces to the device via the SPI interface. Writing to, and reading from, all registers is accomplished using standard SPI read and write commands, in addition to specialized SPI commands.

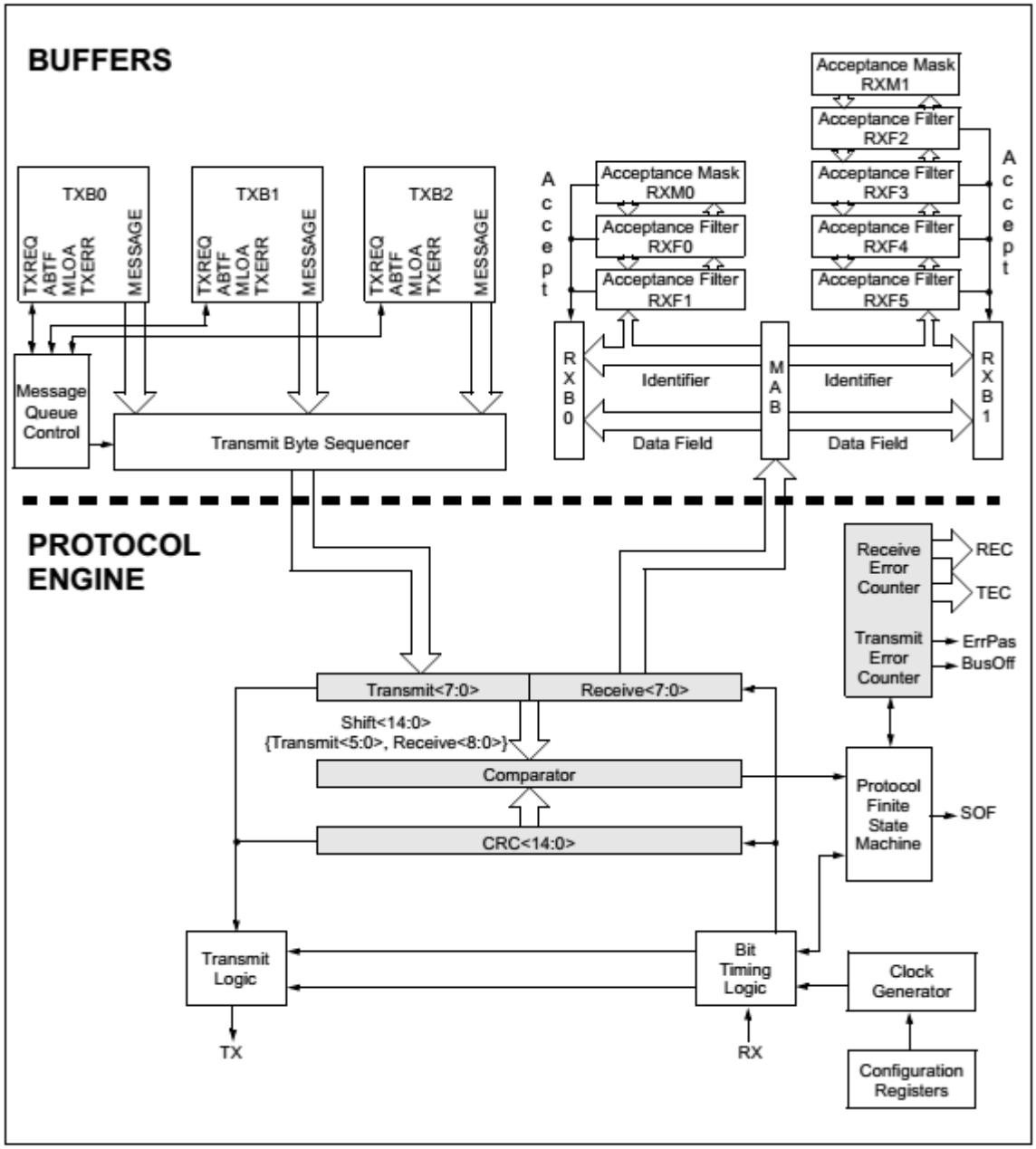
FIGURE 1-1: BLOCK DIAGRAM



1.4 Transmit/Receive Buffers/Masks/Filters

The MCP2515 has three transmit and two receive buffers, two acceptance masks (one for each receive buffer) and a total of six acceptance filters. Figure 1-3 shows a block diagram of these buffers and their connection to the protocol engine.

FIGURE 1-3: CAN BUFFERS AND PROTOCOL ENGINE BLOCK DIAGRAM



3.0 MESSAGE TRANSMISSION

3.1 Transmit Buffers

The MCP2515 implements three transmit buffers. Each of these buffers occupies 14 bytes of SRAM and are mapped into the device memory map.

The first byte, TXBnCTRL, is a control register associated with the message buffer. The information in this register determines the conditions under which the message will be transmitted and indicates the status of the message transmission (see Register 3-1).

Five bytes are used to hold the standard and extended identifiers, as well as other message arbitration information (see Register 3-3 through Register 3-7). The last eight bytes are for the eight possible data bytes of the message to be transmitted (see Register 3-8).

At a minimum, the TXBnSIDH, TXBnSIDL and TXBnDLC registers must be loaded. If data bytes are present in the message, the TXBnDm registers must also be loaded. If the message is to use extended identifiers, the TXBnEIDm registers must also be loaded and the TXBnSIDL.EXIDE bit set.

Prior to sending the message, the MCU must initialize the CANINTE.TXInE bit to enable or disable the generation of an interrupt when the message is sent.

Note:	The TXBnCTRL.TXREQ bit must be clear (indicating the transmit buffer is not pending transmission) before writing to the transmit buffer.

3.2 Transmit Priority

Transmit priority is a prioritization within the MCP2515 of the pending transmittable messages. This is independent from, and not necessarily related to, any prioritization implicit in the message arbitration scheme built into the CAN protocol.

Prior to sending the SOF, the priority of all buffers that are queued for transmission is compared. The transmit buffer with the highest priority will be sent first. For example, if transmit buffer 0 has a higher priority setting than transmit buffer 1, buffer 0 will be sent first.

If two buffers have the same priority setting, the buffer with the highest buffer number will be sent first. For example, if transmit buffer 1 has the same priority setting as transmit buffer 0, buffer 1 will be sent first.

There are four levels of transmit priority. If TXBnCTRL.TXP<1:0> for a particular message buffer is set to `11`, that buffer has the highest possible priority. If TXBnCTRL.TXP<1:0> for a particular message buffer is `00`, that buffer has the lowest possible priority.

3.3 Initiating Transmission

In order to initiate message transmission, the TXBnCTRL.TXREQ bit must be set for each buffer to be transmitted. This can be accomplished by:

- Writing to the register via the SPI write command
- Sending the SPI RTS command
- Setting the $\overline{\text{TXnRTS}}$ pin low for the particular transmit buffer(s) that are to be transmitted

If transmission is initiated via the SPI interface, the TXREQ bit can be set at the same time as the TXP priority bits.

When TXBnCTRL.TXREQ is set, the TXBnCTRL.ABTF, TXBnCTRL.MLOA and TXBnCTRL.TXERR bits will be cleared automatically.

Note:	Setting the TXBnCTRL.TXREQ bit does not initiate a message transmission. It merely flags a message buffer as being ready for transmission. Transmission will start when the device detects that the bus is available.

Once the transmission has completed successfully, the TXBnCTRL.TXREQ bit will be cleared, the CANINTF.TXnIF bit will be set and an interrupt will be generated if the CANINTE.TXnIE bit is set.

If the message transmission fails, the TXBnCTRL.TXREQ will remain set. This indicates that the message is still pending for transmission and one of the following condition flags will be set:

- If the message started to transmit but encountered an error condition, the TXBnCTRL.TXERR and the CANINTF.MERRF bits will be set and an interrupt will be generated on the $\overline{\text{INT}}$ pin if the CANINTE.MERRE bit is set
- If the message is lost, arbitration at the TXBnCTRL.MLOA bit will be set

Note:	If One-shot mode is enabled (CANCTRL.OSM), the above conditions will still exist. However, the TXREQ bit will be cleared and the message will not attempt transmission a second time.

3.4 One-Shot Mode

One-shot mode ensures that a message will only attempt to transmit one time. Normally, if a CAN message loses arbitration, or is destroyed by an error frame, the message is retransmitted. With One-shot mode enabled, a message will only attempt to transmit one time, regardless of arbitration loss or error frame.

One-shot mode is required to maintain time slots in deterministic systems, such as TTCAN.

4.0 MESSAGE RECEPTION

4.1 Receive Message Buffering

The MCP2515 includes two full receive buffers with multiple acceptance filters for each. There is also a separate Message Assembly Buffer (MAB) that acts as a third receive buffer (see Figure 4-2).

4.1.1 MESSAGE ASSEMBLY BUFFER

Of the three receive buffers, the MAB is always committed to receiving the next message from the bus. The MAB assembles all messages received. These messages will be transferred to the RXBn buffers (See Register 4-4 to Register 4-9) only if the acceptance filter criteria is met.

4.1.2 RXB0 AND RXB1

The remaining two receive buffers, called RXB0 and RXB1, can receive a complete message from the protocol engine via the MAB. The MCU can access one buffer, while the other buffer is available for message reception, or for holding a previously received message.

Note:	The entire contents of the MAB is moved into the receive buffer once a message is accepted. This means that, regardless of the type of identifier (standard or extended) and the number of data bytes received, the entire receive buffer is overwritten with the MAB contents. Therefore, the contents of all registers in the buffer must be assumed to have been modified when any message is received.

4.1.3 RECEIVE FLAGS/INTERRUPTS

When a message is moved into either of the receive buffers, the appropriate CANINTF.RXnIF bit is set. This bit must be cleared by the MCU in order to allow a new message to be received into the buffer. This bit provides a positive lockout to ensure that the MCU has finished with the message before the MCP2515 attempts to load a new message into the receive buffer.

If the CANINTE.RXnIE bit is set, an interrupt will be generated on the $\overline{\text{INT}}$ pin to indicate that a valid message has been received. In addition, the associated RXnBF pin will drive low if configured as a receive buffer full pin. See **Section 4.4 "RX0BF and RX1BF Pins"** for details.

4.2 Receive Priority

RXB0, the higher priority buffer, has one mask and two message acceptance filters associated with it. The received message is applied to the mask and filters for RXB0 first.

RXB1 is the lower priority buffer, with one mask and four acceptance filters associated with it.

In addition to the message being applied to the RB0 mask and filters first, the lower number of acceptance filters makes the match on RXB0 more restrictive and implies a higher priority for that buffer.

When a message is received, bits <3:0> of the RXBnCTRL register will indicate the acceptance filter number that enabled reception and whether the received message is a remote transfer request.

4.2.1 ROLLOVER

Additionally, the RXB0CTRL register can be configured such that, if RXB0 contains a valid message and another valid message is received, an overflow error will not occur and the new message will be moved into RXB1, regardless of the acceptance criteria of RXB1.

4.2.2 RXM BITS

The RXBnCTRL.RXM bits set special receive modes. Normally, these bits are cleared to `00` to enable reception of all valid messages as determined by the appropriate acceptance filters. In this case, the determination of whether or not to receive standard or extended messages is determined by the RFXnSIDL.EXIDE bit in the acceptance filter register.

If the RXBnCTRL.RXM bits are set to `01` or `10`, the receiver will only accept messages with standard or extended identifiers, respectively. If an acceptance filter has the RFXnSIDL.EXIDE bit set such that it does not correspond with the RXBnCTRL.RXM mode, that acceptance filter is rendered useless. These two modes of RXBnCTRL.RXM bits can be used in systems where it is known that only standard or extended messages will be on the bus.

If the RXBnCTRL.RXM bits are set to `11`, the buffer will receive all messages, regardless of the values of the acceptance filters. Also, if a message has an error before the EOF, that portion of the message assembled in the MAB before the error frame will be loaded into the buffer. This mode has some value in debugging a CAN system and would not be used in an actual system environment.

MCP2515

4.5 Message Acceptance Filters and Masks

The message acceptance filters and masks are used to determine if a message in the message assembly buffer should be loaded into either of the receive buffers (see Figure 4-5). Once a valid message has been received into the MAB, the identifier fields of the message are compared to the filter values. If there is a match, that message will be loaded into the appropriate receive buffer.

4.5.1 DATA BYTE FILTERING

When receiving standard data frames (11-bit identifier), the MCP2515 automatically applies 16 bits of masks and filters normally associated with extended identifiers to the first 16 bits of the data field (data bytes 0 and 1). Figure 4-4 illustrates how masks and filters apply to extended and standard data frames.

Data byte filtering reduces the load on the MCU when implementing Higher Layer Protocols (HLPs) that filter on the first data byte (e.g., DeviceNet™).

4.5.2 FILTER MATCHING

The filter masks (see Register 4-14 through Register 4-17) are used to determine which bits in the identifier are examined with the filters. A truth table is shown in Table 4-2 that indicates how each bit in the identifier is compared to the masks and filters to determine if the message should be loaded into a receive buffer. The mask essentially determines which bits to apply the acceptance filters to. If any mask bit is set to a zero, that bit will automatically be accepted, regardless of the filter bit.

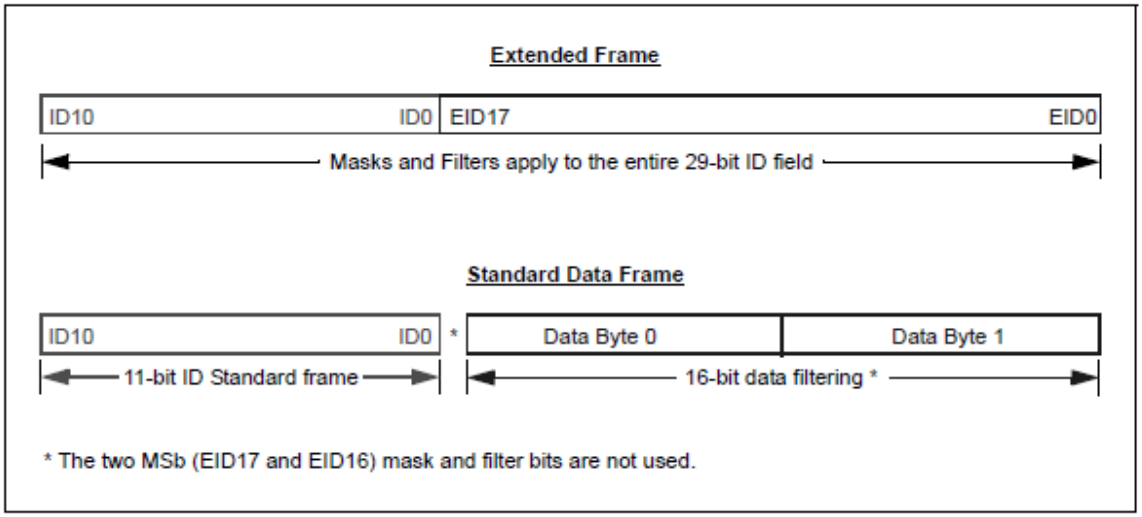
TABLE 4-2: FILTER/MASK TRUTH TABLE

Mask Bit n	Filter Bit n	Message Identifier bit	Accept or Reject bit n
0	x	x	Accept
1	0	0	Accept
1	0	1	Reject
1	1	0	Reject
1	1	1	Accept

Note: x = don't care

As shown in the receive buffers block diagram (Figure 4-2), acceptance filters RXF0 and RXF1 (and filter mask RXM0) are associated with RXB0. Filters RXF2, RXF3, RXF4, RXF5 and mask RXM1 are associated with RXB1.

FIGURE 4-4: MASKS AND FILTERS APPLY TO CAN FRAMES



 © 2005 Microchip Technology Inc.

Annexe 2 - Listing du fichier BusTer.h

```
// File     : BusTerUtils.h

#ifndef _BUSTER_UTILS_H_
#define _BUSTER_UTILS_H_

#include "Arduino.h"

#ifndef INT32U
#define INT32U unsigned long
#endif

#ifndef INT8U
#define INT8U byte
#endif

// Default Modbus port
#define BUSTER_MODBUS_PORT 502

// Define the pins used on Arduino

// Digital inputs
#define BUSTER_PIN_IN_0    40
#define BUSTER_PIN_IN_1    41
#define BUSTER_PIN_IN_2    38
#define BUSTER_PIN_IN_3    39
#define BUSTER_PIN_IN_4    36
#define BUSTER_PIN_IN_5    37
#define BUSTER_PIN_IN_6    34
#define BUSTER_PIN_IN_7    35

// Digital outputs
#define BUSTER_PIN_OUT_0 48
#define BUSTER_PIN_OUT_1 49
#define BUSTER_PIN_OUT_2 46
#define BUSTER_PIN_OUT_3 47
#define BUSTER_PIN_OUT_4 44
#define BUSTER_PIN_OUT_5 45
#define BUSTER_PIN_OUT_6 42
#define BUSTER_PIN_OUT_7 43

// Dip switch
#define BUSTER_PIN_DIPSWITCH_1 32
#define BUSTER_PIN_DIPSWITCH_2 33
#define BUSTER_PIN_DIPSWITCH_3 30
#define BUSTER_PIN_DIPSWITCH_4 31
#define BUSTER_PIN_DIPSWITCH_5 28
#define BUSTER_PIN_DIPSWITCH_6 29
#define BUSTER_PIN_DIPSWITCH_7 26
#define BUSTER_PIN_DIPSWITCH_8 27

// Status led
#define BUSTER_PIN_STATUS_LED_GREEN 24
#define BUSTER_PIN_STATUS_LED_RED 25
```

```
enum BUSTER_AnalogInput
{
    BUSTER_ANALOGINPUT_0, BUSTER_ANALOGINPUT_1,
    BUSTER_ANALOGINPUT_2, BUSTER_ANALOGINPUT_3
};

// ***************************************************************************
// Function name: BusTer_InitInputOutput
// Descriptions:  Init the input and output of the arduino
// ***************************************************************************
void BusTer_InitInputOutput();

// ***************************************************************************
// Function name: BusTer_AnalogRead
// Descriptions:  Read an analog input corresponding to the input parameter.
// return value:  returns the input voltage in mV, range 0..5000.
// ***************************************************************************
int BusTer_AnalogRead(BUSTER_AnalogInput input);

// ***************************************************************************
// Function name: BusTer_AnalogWrite
// Descriptions:  Write the 4 analog outputs
//                Values are in [mV] in the range of 0..5000 mV
// ***************************************************************************
void BusTer_AnalogWrite(int output0, int output1, int output2, int output3);

// ************************** API CAN for Arduino ****************************

// Can bit rates
enum BUSTER_CanSpeed
{
    BUSTER_CAN_5KBPS = 1, BUSTER_CAN_10KBPS, BUSTER_CAN_20KBPS,
    BUSTER_CAN_31K25BPS, BUSTER_CAN_33KBPS, BUSTER_CAN_40KBPS,
    BUSTER_CAN_50KBPS, BUSTER_CAN_80KBPS, BUSTER_CAN_83K3BPS,
    BUSTER_CAN_95KBPS, BUSTER_CAN_100KBPS, BUSTER_CAN_125KBPS,
    BUSTER_CAN_200KBPS, BUSTER_CAN_250KBPS, BUSTER_CAN_500KBPS,
    BUSTER_CAN_1MBPS
};

// Can formats
enum BUSTER_CanType
{
    BUSTER_Can2_0A = 0, BUSTER_Can2_0B
};

// Receive can buffer registers
enum BUSTER_CanRxb
{
    BUSTER_Rxb0, BUSTER_Rxb1
};

// Receive mask registers
enum BUSTER_CanRxm
{
    BUSTER_Rxm0, BUSTER_Rxm1
};
```

```
// Receive filter registers
enum BUSTER_CanRxf
{
    BUSTER_Rxf0, BUSTER_Rxf1, BUSTER_Rxf2, BUSTER_Rxf3, BUSTER_Rxf4,
    BUSTER_Rxf5
};

// Transmit buffers registers
enum BUSTER_CanTxb
{
    BUSTER_Txb0 = 0x30, BUSTER_Txb1 = 0x40, BUSTER_Txb2 = 0x50
};

// ***************************************************************************
// Function name:           BusTer_CanInit
// Descriptions:            Init the MCP2515 can controller with the canSpeed
//                          Set the controller in configuration mode
// ***************************************************************************
void  BusTer_CanInit(BUSTER_CanSpeed canSpeed);

// ***************************************************************************
// Function name:       BusTer_CanSetMask
// Parameter rxMask:    the mask register to set
// Parameter type:      Specification CAN_2.0A or CAN_2.0B
// Parameter ulData:    mask to init
// Descriptions:        init canid masks, returns true if success
// ***************************************************************************
bool BusTer_CanSetMask(BUSTER_CanRxm rxMask, BUSTER_CanType type,
    INT32U ulData);

// ***************************************************************************
// Function name:      BusTer_CanSetFilter
// Parameter rxFilter: the filter register to set
// Parameter type:     Message format CAN_2.0A or CAN_2.0B
// Parameter ulData:   filter to init
// Descriptions:       init canid filters, returns true if success
// ***************************************************************************
bool BusTer_CanSetFilter(BUSTER_CanRxf rxFilter, BUSTER_CanType type,
    INT32U ulData);

// ***************************************************************************
// Function name:      BusTer_CanActivateNormalMode
// Descriptions:       Set the MCP2515 in normal operation mode
//                     returns true if success
// ***************************************************************************
bool BusTer_CanActivateNormalMode();

// ***************************************************************************
// Function name:    BusTer_CanSend
// parameter Txbuf : Number of the transmit buffer (ENUM)
// parameter type :  Specification CAN_2.0A or CAN_2.0B (ENUM)
// parameter arb :   arbitration field
// parameter len :   length of the data buffer
// parameter *buf :  data
// Descriptions:     write message to send buffer.
// ***************************************************************************
void BusTer_CanSend(BUSTER_CanTxb Txbuf, BUSTER_CanType type, INT32U arb,
    INT8U len, INT8U *buf);
```

```
// ***************************************************************************
// Function name: BusTer_CanSendReady
// Descriptions:  returns true if the transmit buffer (txbuf_n) is
//                ready to send a new message
// ***************************************************************************
bool  BusTer_CanSendReady(BUSTER_CanTxb txbuf_n);

// ***************************************************************************
// Function name:       BusTer_CanMessageReceived
// parameter RxbBuffer: buffer to check if a message is received
// Descriptions:        returns true if a message has been received
//***************************************************************************
bool  BusTer_CanMessageReceived(BUSTER_CanRxb RxbBuffer);

// ***************************************************************************
// Function name:        BusTer_CanRead
// parameter RxbBuffer : Read in the following buffer
// parameter arb :       return the arbitration field received
// parameter len :       return the length of the data received
// parameter buf[] :     adress of the buffer to put the data
// Descriptions:         reads message buf and can bus source arb,
//                       Returns true if success
// ***************************************************************************
bool BusTer_CanRead(BUSTER_CanRxb RxbBuffer, INT32U *arb, INT8U *len,
    INT8U *buf);

// ***************************************************************************
// Function name:           BusTer_CanHasError
// Descriptions:            if can communication in error, returns true
// ***************************************************************************
bool  BusTer_CanHasError();

// ***************************************************************************
// Function name:           BusTer_CanClearError
// Descriptions:            clear can error
// ***************************************************************************
void BusTer_CanClearError();

#endif
```